

Table 1: Execution time results

Benchmark	C++	C#	C	Vala
mandelbrot	14.48	47.40	12.38	13.05
partialSums	32.68	56.58	35.11	34.92
recursive	12.87	28.15	8.38	8.61
binaryTrees	27.53	42.62	21.56	30.75
sumFile	16.90	23.70	14.18	15.24
fannkuch	11.18	26.59	12.26	14.23
spectralNorm	32.77	46.58	32.83	36.84
nsieve	25.49	29.31	26.09	25.70
nBody	28.33	43.55	26.02	28.06

Mixing Vala and Genie

You can seamlessly mix Vala and Genie files in an application, since both are converted into the intermediate C code by *valac*. It means that you can write a function in Genie and use that in Vala; write a class in Vala and use its properties/methods in Genie, and so on. Below we have a small example that (in Vala) derives a class from a superclass defined in Genie, and invokes inherited and class methods. Compile with: `valac -o valagenie classvala.vala classgenie.gs`.

classgenie.gs:

```
[indent=4]
class Genie: Object
def printInfoGenie(info: string)
    print "\n Genie:: %s\n", info
```

classvala.vala:

```
class Vala: Genie
{
    void printInfoVala(string info) {
        stdout.printf("\n Vala:: %s\n", info);
    }
    public static void main() {
        var ovala = new Vala();
```

```
        ovala.printInfoVala("Calling method of Vala class from Vala class.");
        ovala.printInfoGenie("Calling method of Genie class from Vala
class.");
    }
}
```

Performance notes

The performance of Vala and Genie programs is equal or almost equivalent to C programs, since *valac* compiles your code into an intermediate form of very efficient GLib- and GObject-based C code. There is an independent project on Google Code to implement *The Computer Language Benchmarks Game* in Vala, and to compare its performance against C, C++ and C#. Table 1 shows the execution time results published on the benchmarks project page using GCC-4.3.4, Mono C# compiler 2.4.2.3 and *valac*-0.7.6.

You can see that Vala's performance numbers are close to those of C, in all the benchmarks.

Vala and Genie are modern programming languages that promise high productivity, flexibility and native performance without requiring a large number of run-time libraries or virtual machines. These languages provide GNOME (and even embedded-systems) developers some excellent programming alternatives to writing programs in the venerable C language. Genie, in particular, looks like a very attractive way for Python and similar scripting-language developers to gain native-code speed for their applications. 

References

- GObject Wikipedia page: <http://en.wikipedia.org/wiki/GObject>
- Vala homepage: <http://live.gnome.org/Vala>
- Valadoc homepage: <http://live.gnome.org/Valadoc>
- Libgee homepage: <http://live.gnome.org/Libgee>
- Genie homepage: <http://live.gnome.org/Genie>
- Genie tutorial from the founder of Puppy Linux: <http://bkhome.org/genie/index.html>
- Collection of Vala and Genie examples: <http://code.valaide.org>
- Vala benchmarks: <http://code.google.com/p/Vala-benchmarks>

By: Ankur Kumar Sharma

The author is a software developer and researcher with industry experience of over eight years in various areas like kernel/hardware programming, systems development, automation tools development, R&D, etc. He is now the founder of RichNusGeeks Consulting, an open source consulting and training start-up. He likes to play classic rock music on his guitar and croon along; read self-help books, write, and explore stuff that interests him in his spare time. He blogs at <http://www.richnusgeeks.wordpress.com>.

Managing IT is **simpler**, and **more cost effective** using Open Source

Read LINUX For You: Asia's Leading Open Source Magazine

